Lab - Creating a Custom MySQL Image
To Begin with the Lab

Summary
This lab demonstrates how to create a **custom MySQL Docker image** that automatically initializes the database during container startup. Instead of manually creating the database, users, tables, and sample data after launching the container, all these tasks are executed automatically using a SQL initialization script included in the Docker image.

Understanding
The official MySQL Docker image allows automatic database initialization by executing any SQL files placed inside the **/docker-entrypoint-initdb.d** directory. When the MySQL container starts for the first time, it runs all SQL scripts in this directory.
By creating a **custom Docker image**, you can preconfigure:

- Database creation
- User creation
- Granting permissions
- Table creation
- Sample data insertion

As a result, every new container created from this image is fully initialized and ready to use without any manual SQL commands.
Example:
Instead of manually running:

- Create Database
- Create User
- Grant Privileges
- Create Table
- Insert Data

the container performs all these steps automatically during startup.

Lab Steps
Step 1: Create a Project Folder

- Create an empty folder for the **MySQL** Docker project.
- Store all required files inside this folder.

Example:
MySQL-Docker/

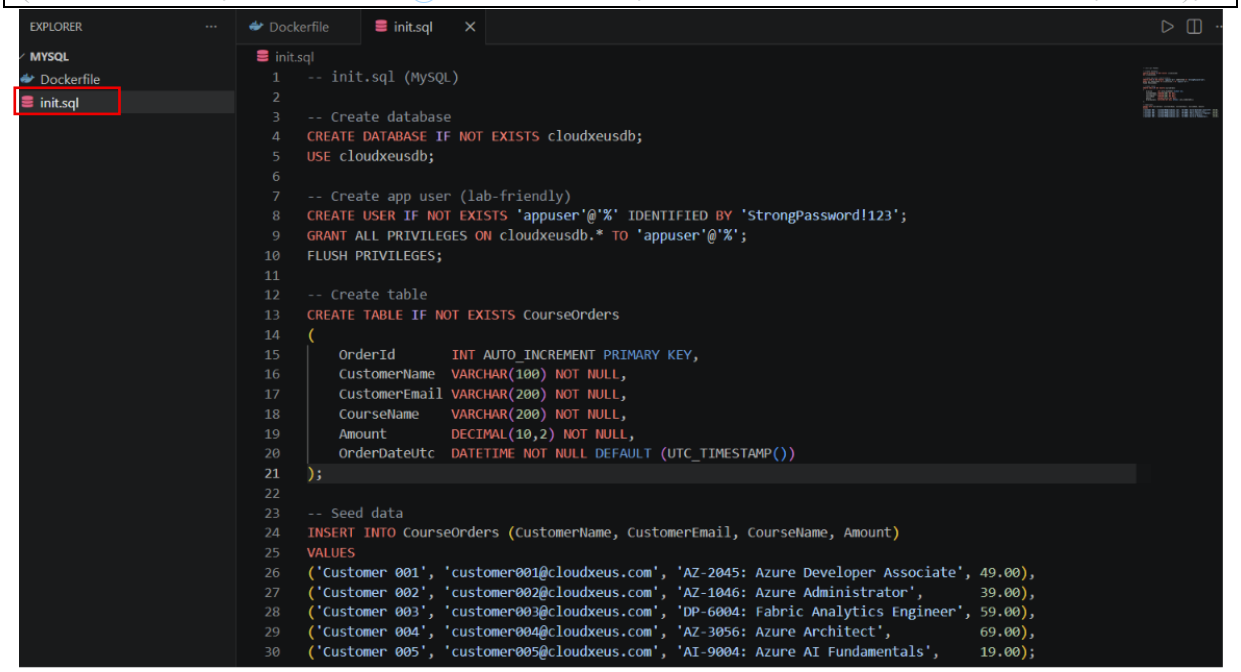
```
├── Dockerfile
└── init.sql
```

Step 2: Create the SQL Initialization Script

- Create a file named: init.sql
- Paste the code in init,sql.

```
-- init.sql (MySQL)

-- Create database
CREATE DATABASE IF NOT EXISTS cloudxeusdb;
USE cloudxeusdb;

-- Create app user (lab-friendly)
CREATE USER IF NOT EXISTS 'appuser'@'%' IDENTIFIED BY 'StrongPassword!123';
```

```
GRANT ALL PRIVILEGES ON cloudxeusdb.* TO 'appuser'@'%';
FLUSH PRIVILEGES;

-- Create table
CREATE TABLE IF NOT EXISTS CourseOrders
(
   OrderId       INT AUTO_INCREMENT PRIMARY KEY,
   CustomerName  VARCHAR(100) NOT NULL,
   CustomerEmail VARCHAR(200) NOT NULL,
   CourseName    VARCHAR(200) NOT NULL,
   Amount        DECIMAL(10,2) NOT NULL,
   OrderDateUtc  DATETIME NOT NULL DEFAULT (UTC_TIMESTAMP())
);

-- Seed data
INSERT INTO CourseOrders (CustomerName, CustomerEmail, CourseName, Amount)
VALUES
('Customer 001', 'customer001@cloudxeus.com', 'AZ-2045: Azure Developer Associate',
49.00),
('Customer 002', 'customer002@cloudxeus.com', 'AZ-1046: Azure Administrator',
39.00),
('Customer 003', 'customer003@cloudxeus.com', 'DP-6004: Fabric Analytics Engineer',
59.00),
('Customer 004', 'customer004@cloudxeus.com', 'AZ-3056: Azure Architect', 69.00),
('Customer 005', 'customer005@cloudxeus.com', 'AI-9004: Azure AI Fundamentals', 19.00);
```



Step 3: Create the Dockerfile

- Create a file named: Dockerfile
- Paste the code in **Dockerfile**

```
FROM mysql:8.4
```

```
COPY init.sql /docker-entrypoint-initdb.d/01-init.sql
```

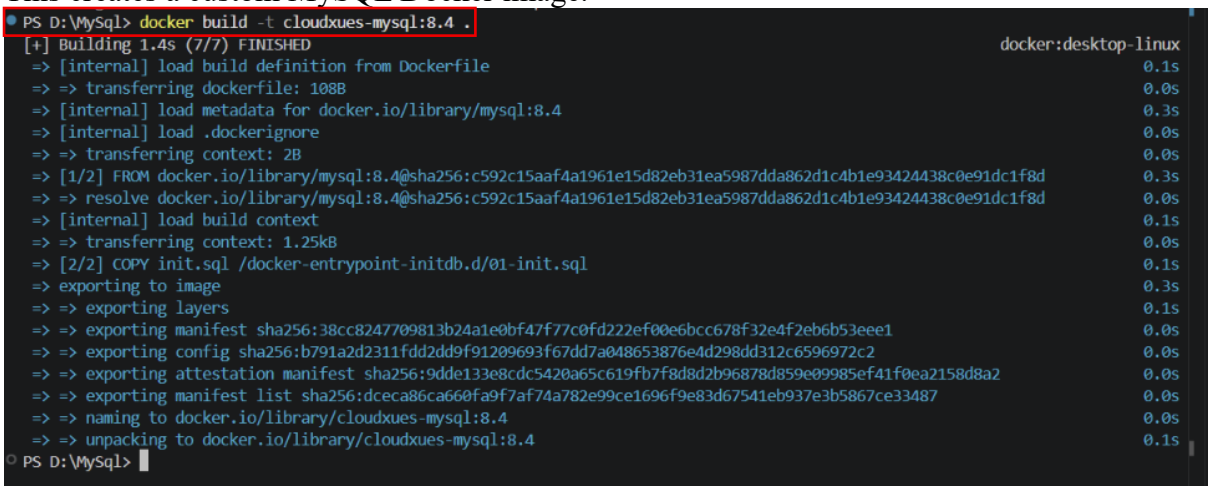
Step 4: Build the Custom Docker Image

Open the terminal inside the project folder.

Run:

```
docker build -t cloudxues-mysql:8.4 .
```

This creates a custom MySQL Docker image.

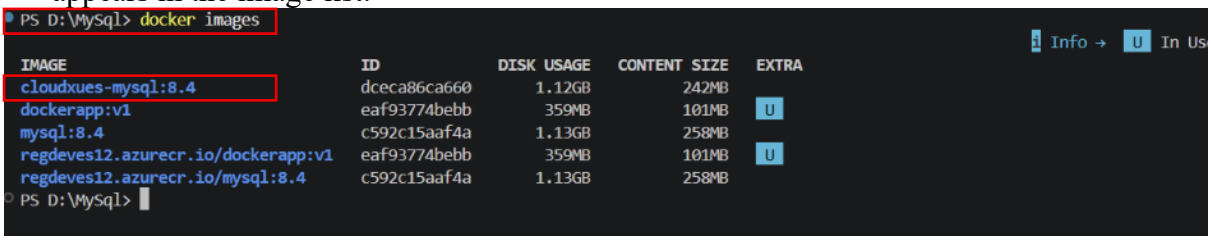


Step 5: Verify the Image

- Run:

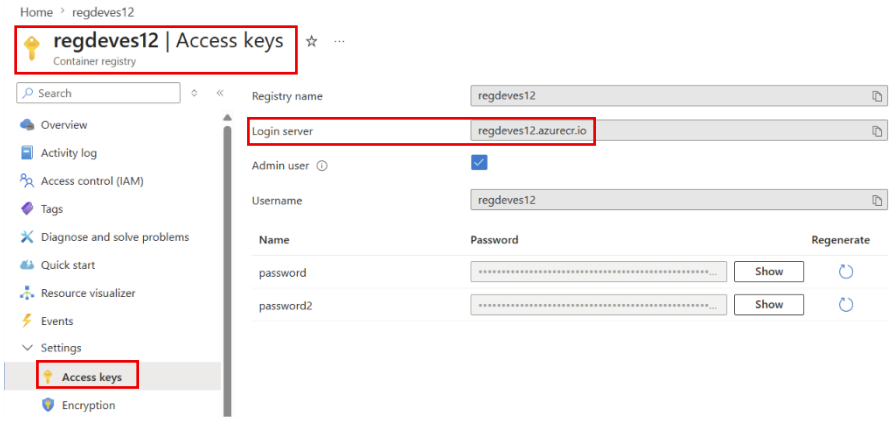
```
docker images
```

- Verify that:
- Cloudxeus-mysql:8.4
- appears in the image list.



Step 6: Tag the Image for Azure Container Registry

- Go to the **Azure container Registry.**
- Navigate to **Access keys.**
- Copy the **Login server.**



Run:

```
docker tag cloudxues-mysql:8.4 regdeves12.azurecr.io/cloudxues-mysql:8.4
```

```
PS D:\MySql> docker tag cloudxues-mysql:8.4 regdeves12.azurecr.io/cloudxues-mysql:8.4
PS D:\MySql>
```

Step 7: Push the Image to Azure Container Registry

Run:

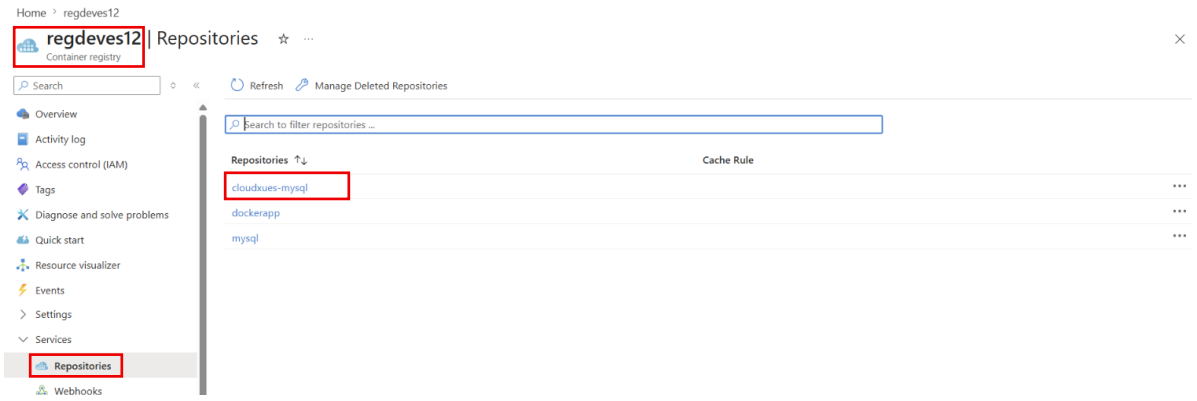
```
docker push regdeves12.azurecr.io/Cloudxeus-mysql:8.4
```

Wait until the upload completes successfully.

```
PS D:\MySql> docker push regdeves12.azurecr.io/cloudxues-mysql:8.4
The push refers to repository [regdeves12.azurecr.io/cloudxues-mysql]
6396417485a7: Mounted from mysql
38f0fce66536: Mounted from mysql
e86fa202d2cb: Mounted from mysql
c2fb254fc2f4: Mounted from mysql
54fb0f063fe4: Pushed
392f62f0c6a5: Mounted from mysql
5f3f9542983a: Mounted from mysql
06a8291f1ecf: Mounted from mysql
4c0beac24bd0: Mounted from mysql
f6d6b5152082: Pushed
4ec1fc2c51e2: Mounted from mysql
f89150c047c1: Mounted from mysql
8.4: digest: sha256:dceca86ca660fa9f7af74a782e99ce1696f9e83d67541eb937e3b5867ce33487 size: 856
PS D:\MySql>
```

Step 8: Verify the Image in Azure Container Registry

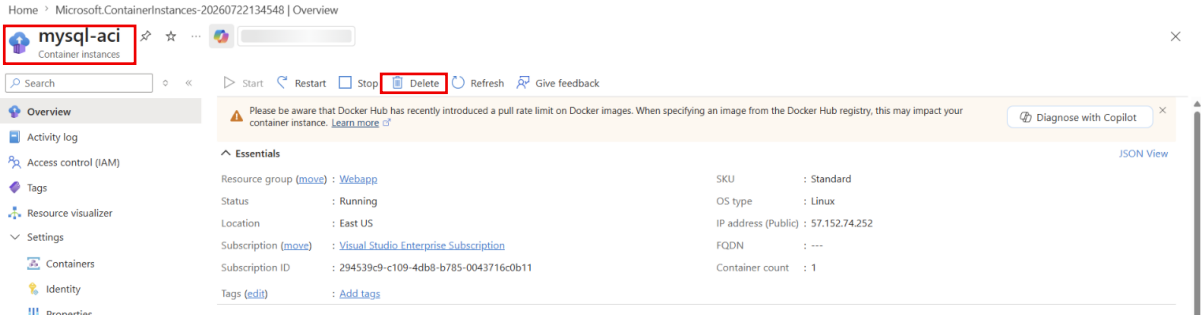
- Open **Azure Container Registry**.
- Select **Repositories**.
- Verify that: Cloudxeus-mysql:8.4 is available.



Step 9: Delete the Existing MySQL Container Instance

- Open **Azure Portal**.

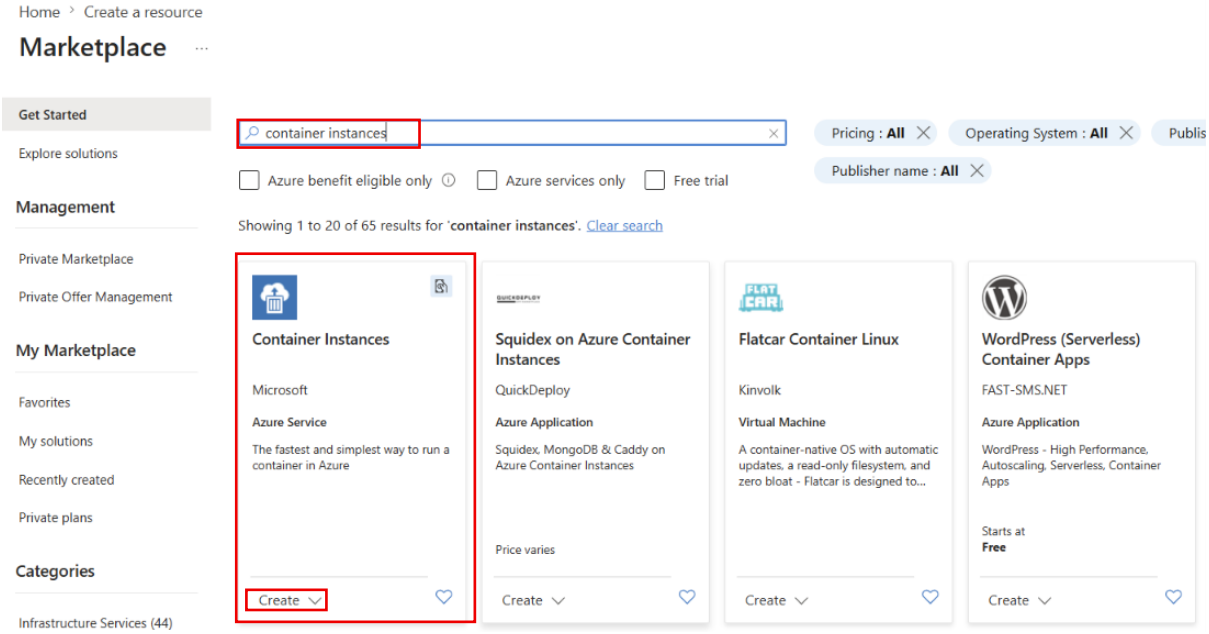
- Navigate to the existing **Container Instance**.
- Click **Delete**.



- Confirm the deletion.

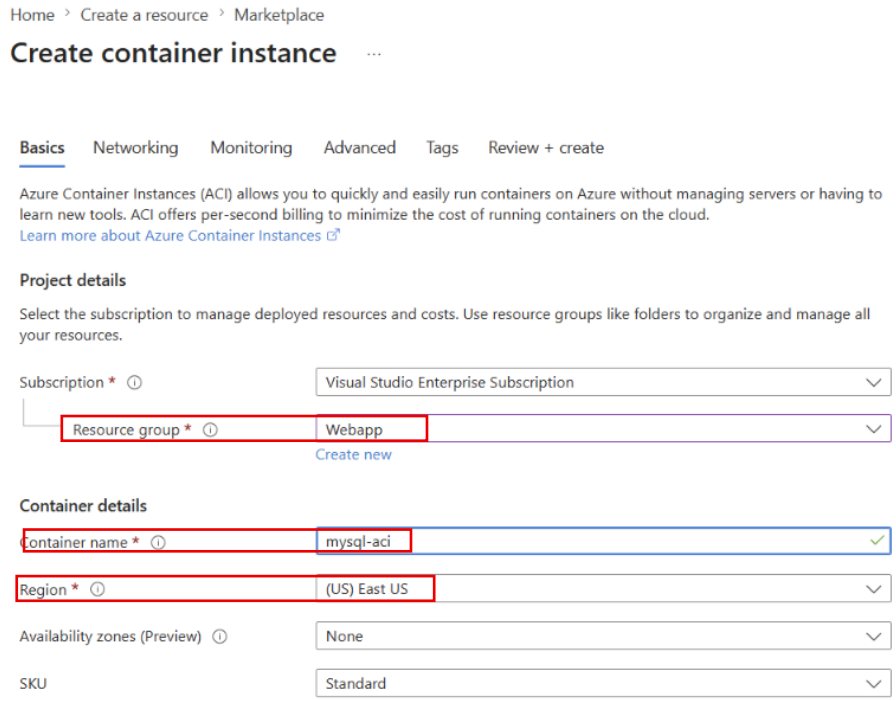
Step 10: Create a New Azure Container Instance

- Click **Create Resource**.
- Search for **Container Instances**.
- Click **Create**.



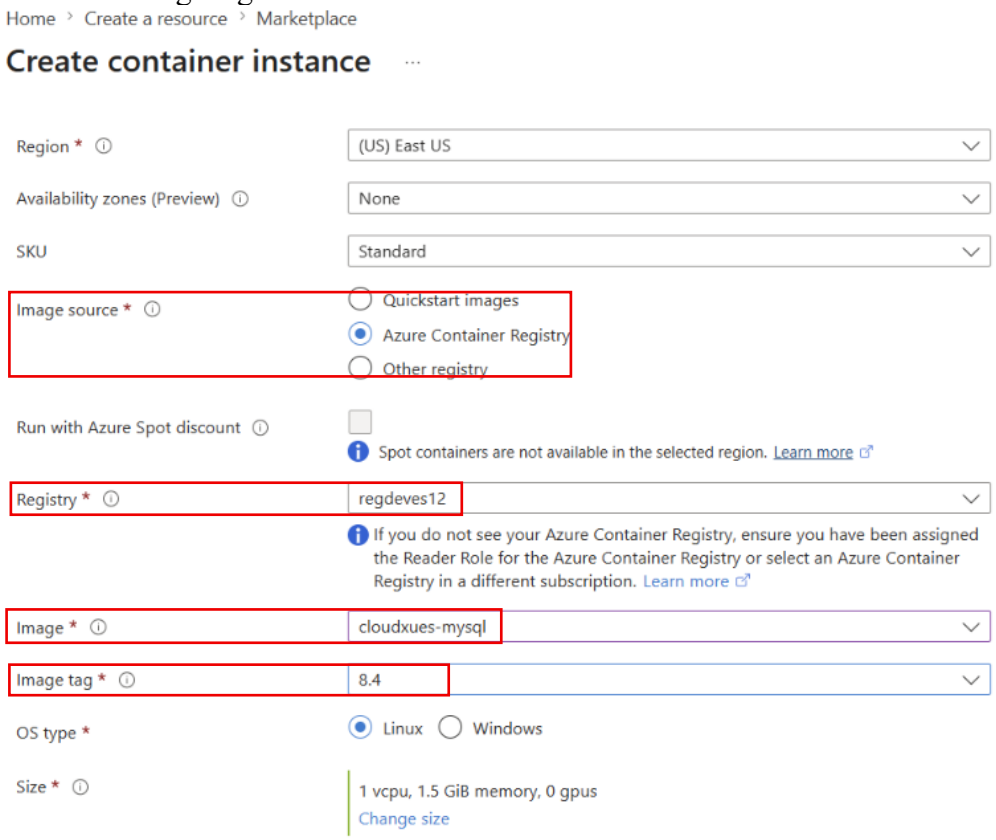
Step 11: Configure Basic Settings

- Select the existing **Resource Group**.
- Enter a **Container Name**.
- Select the **Region** (Example: East US).



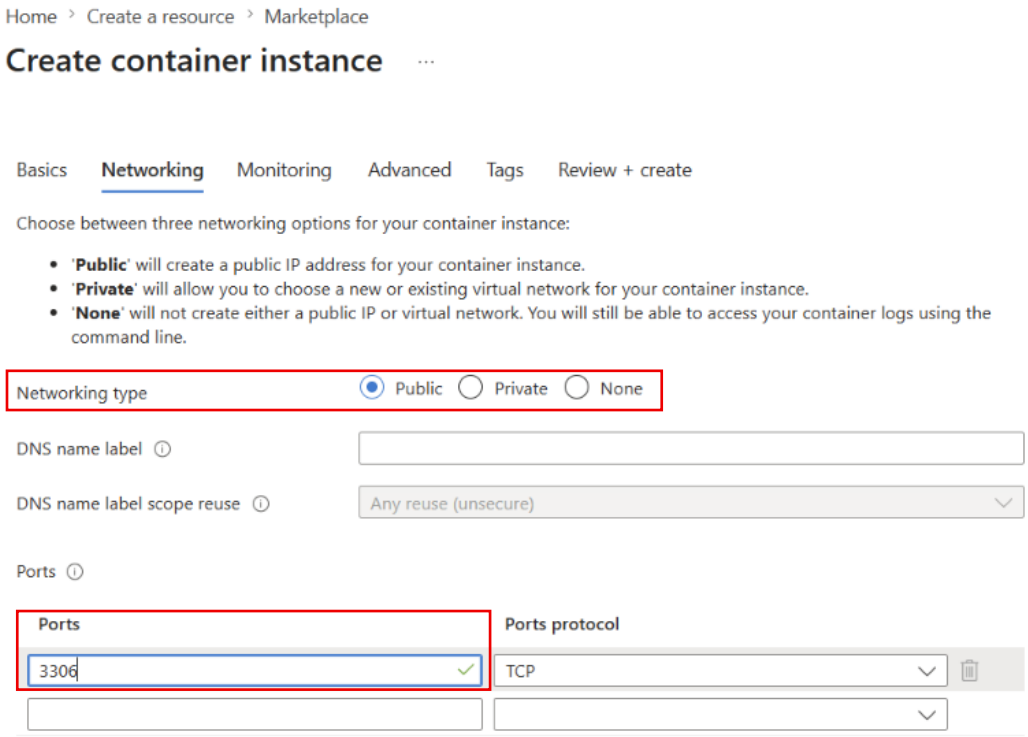
Step 12: Select the Custom Image

- Set **Image Source** to **Azure Container Registry**.
- Select your **Azure Container Registry**.
- Choose the repository: **cloudxues-mysql.**
- Select Image tag **8.4**



Step 13: Configure Networking

- Enable **Public IP** (for the lab).
- Expose **TCP Port 3306**.

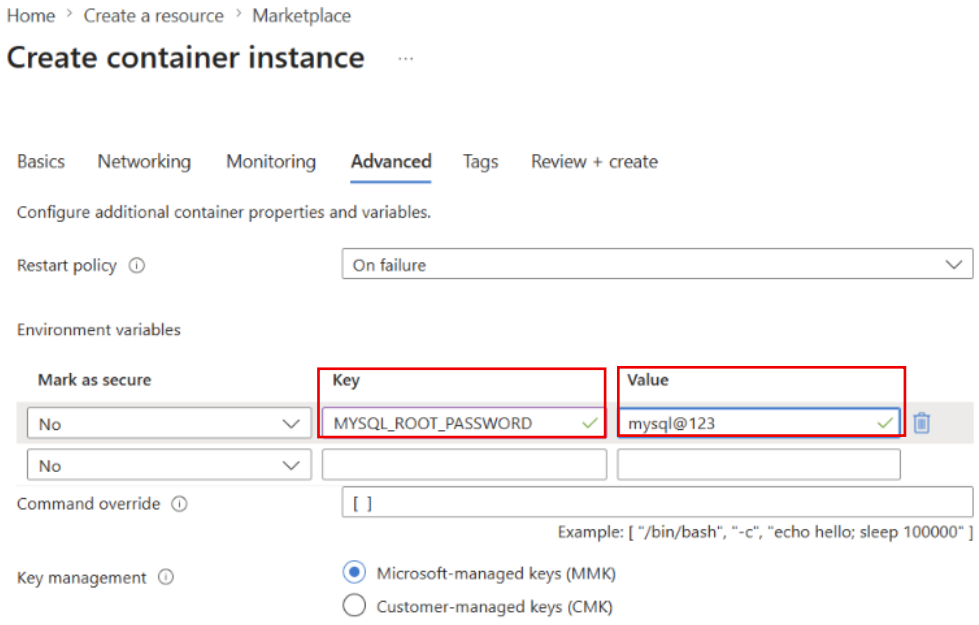


Step 14: Configure Environment Variables

Add the required MySQL environment variable:

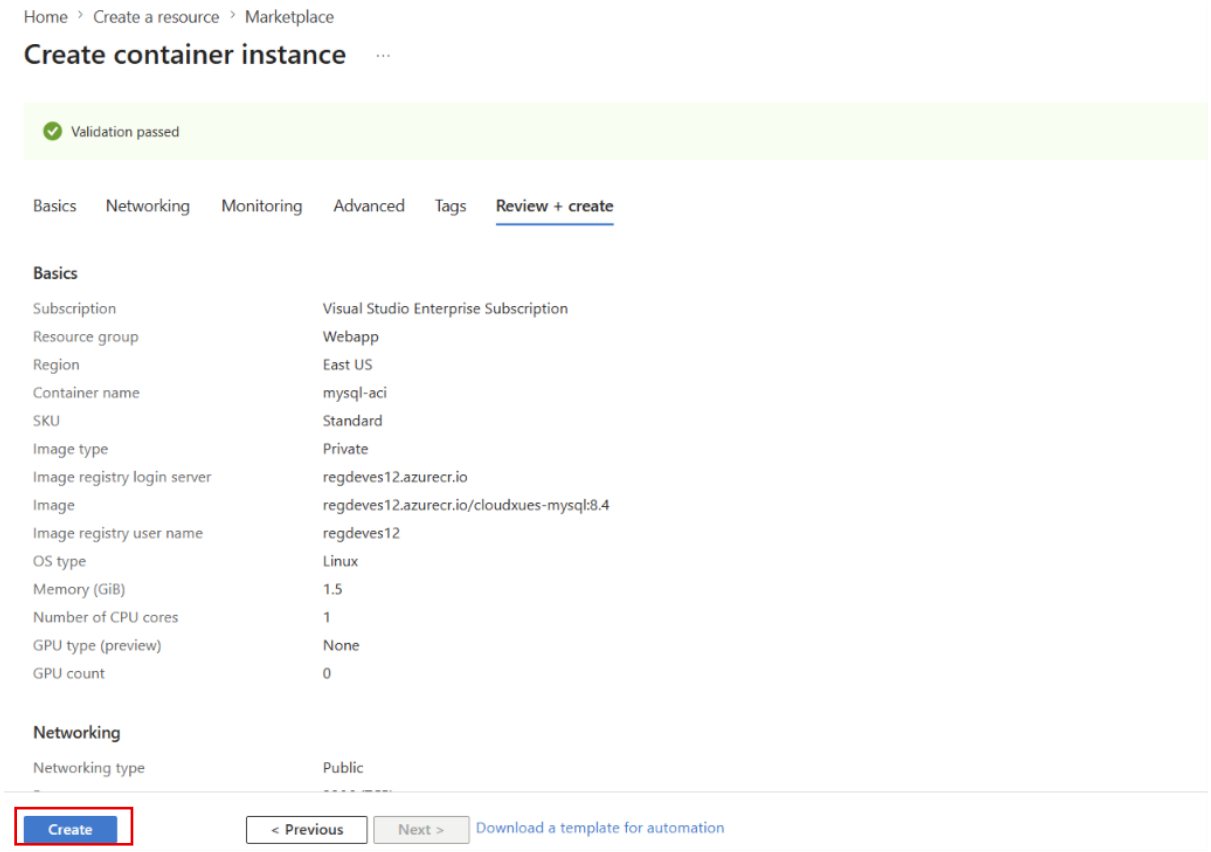
Variable	Value
MYSQL_ROOT_PASSWORD	**mysql@123**

Mark the password as **Secure**.

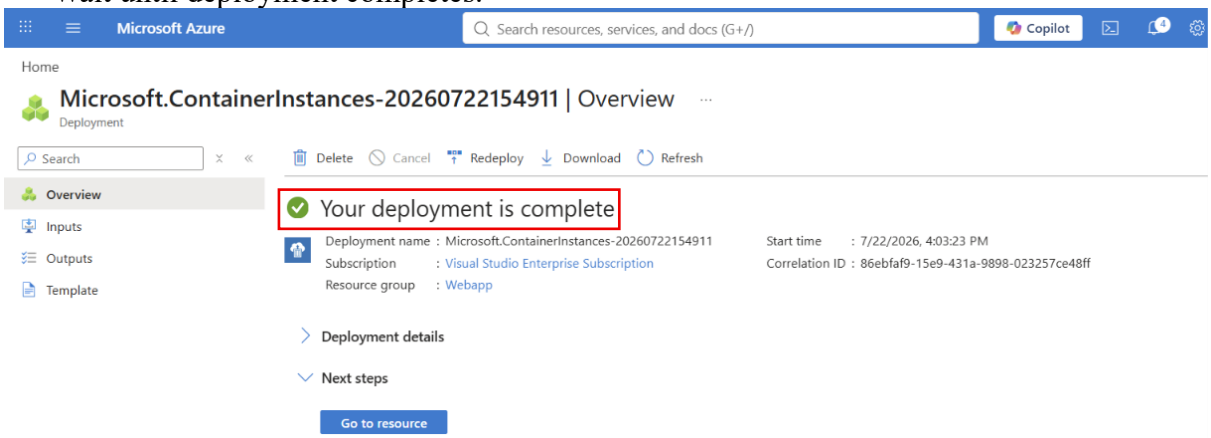


Step 15: Review and Deploy

- Click **Review + Create**.
- Validate the configuration.
- Click **Create**.



- Wait until deployment completes.



Step 16: Verify the Container

- Open the deployed **Container Instance**.
- Navigate to **Containers**.
- Verify the container state is: Running